



Avaliação AV1 - Lógica de Programação

Equipe:

Instruções:

- (1) Cada equipe deve enviar um email para o professor contendo o desenvolvimento das questões e os membros da equipe.
 - (2) Cada equipe deve ler atentamente os textos motivadores e os requisitos dos projetos propostos.
 - (3) Em caso de dúvidas a equipe deve entrar em contato com o professor por email caso não seja o dia da aula presencial
 - (4) As datas e critérios devem ser respeitadas
 - (5) O Atual projeto da AV1 tem valor 10 pontos.
-

Questão 11

(Valor: 1,5 pontos)

O **Algoritmo de Euclides** é um método eficiente para calcular o Máximo Divisor Comum (MDC) entre dois números inteiros, baseado na seguinte propriedade: o MDC entre dois números a e b é igual ao MDC entre b e o resto da divisão de a por b . Repetindo esse processo, o resto vai diminuindo até chegar a 0, e o MDC será o último divisor utilizado (ou seja, o valor de a no momento em que b se torna 0). Formalmente:

$$mdc(a, b) = \begin{cases} a, & \text{se } b = 0 \\ mdc(b, a \bmod b), & \text{se } b \neq 0 \end{cases}$$



Exemplo: para calcular $mdc(48, 18)$: $mdc(48, 18) \rightarrow mdc(18, 12) \rightarrow mdc(12, 6) \rightarrow mdc(6, 0) = 6$. Logo, $mdc(48, 18) = 6$. Essa definição recursiva deve ser usada diretamente na implementação da função `mdc`.

Implemente em C as seguintes funções:

```
1 int mdc(int a, int b);  
2 int mmc(int a, int b);
```

- `mdc`: calcula **recursivamente** o Máximo Divisor Comum entre dois números inteiros positivos, utilizando o Algoritmo de Euclides.
- `mmc`: calcula o Mínimo Múltiplo Comum entre dois números inteiros positivos, **reutilizando a função `mdc`** (lembre-se de que $mmc(a, b) = \frac{a \times b}{mdc(a, b)}$).

A função `main` deve ler dois números inteiros positivos e exibir o valor do MDC e do MMC entre eles.

Questão 12

(Valor: 1,5 pontos)

Bubble Sort: percorre o vetor repetidamente comparando cada par de elementos adjacentes e trocando-os de posição quando estiverem fora de ordem. A cada passagem completa pelo vetor, o maior elemento ainda não ordenado é "borbulhado" até sua posição final, ao final do vetor. O processo se repete até que nenhuma troca seja mais necessária.

Exemplo: $[5, 2, 4, 1] \rightarrow [2, 4, 1, 5] \rightarrow [2, 1, 4, 5] \rightarrow [1, 2, 4, 5]$.

Insertion Sort: constrói o vetor ordenado um elemento por vez. A cada passo, toma-se o próximo elemento do vetor original e o insere na posição correta dentro da parte já ordenada, deslocando os elementos maiores uma posição à frente para abrir espaço.

Exemplo: $[5, 2, 4, 1] \rightarrow [2, 5, 4, 1] \rightarrow [2, 4, 5, 1] \rightarrow [1, 2, 4, 5]$.

Busca Binária: é um método de busca eficiente que funciona apenas em vetores **já ordenados**. Compara-se o valor procurado com o elemento do **meio** do vetor: se forem iguais, o elemento foi encontrado; se o valor procurado for menor, a busca continua apenas na metade esquerda do vetor; se for maior, continua-se na metade direita. Esse processo é repetido (dividindo o intervalo de busca pela metade a cada passo) até encontrar o elemento ou até o intervalo de busca ficar vazio, o que indica que o valor não está presente no vetor.

Implemente em C as seguintes funções, operando sobre vetores de inteiros:



```
1 void ordenar(int v[], int n);  
2 int buscaBinaria(int v[], int n, int chave);
```

- **ordenar**: ordena o vetor em ordem crescente (implemente o algoritmo *Bubble Sort* ou *Insertion Sort*), alterando o vetor original por meio de passagem por referência.
- **buscaBinaria**: realiza busca binária em um vetor **já ordenado**, retornando o índice do elemento procurado, ou -1 caso não seja encontrado.

A função **main** deve ler N valores inteiros ($N \leq 50$, informado pelo usuário), preencher um vetor, ordená-lo usando **ordenar**, exibir o vetor ordenado e, em seguida, solicitar um valor a ser buscado, exibindo sua posição (ou uma mensagem informando que não foi encontrado).

Questão 13

(Valor: 2,0 pontos)

Implemente em C as seguintes funções, para uma matriz quadrada de inteiros de dimensão $N \times N$ ($N \leq 10$):

```
1 void transposta(int m[10][10], int t[10][10], int n);  
2 int traco(int m[10][10], int n);  
3 int eSimetrica(int m[10][10], int n);
```

- **transposta**: calcula a matriz transposta de **m** e armazena em **t**.
- **traco**: calcula o traço da matriz (soma dos elementos da diagonal principal).
- **eSimetrica**: retorna 1 se a matriz for simétrica ($m[i][j] = m[j][i]$ para todo i, j) e 0 caso contrário. **Reutilize a função transposta** na sua implementação.

A função **main** deve ler a dimensão N e os elementos da matriz, calcular e exibir a transposta, o traço e informar se a matriz é simétrica.



Questão 14

(Valor: 2,5 pontos)

Implemente em C as seguintes funções, para matrizes quadradas de inteiros de dimensão $N \times N$ ($N \leq 10$):

```
1 void multiplicarMatrizes(int a[10][10], int b[10][10], int
   resultado[10][10], int n);
2 void extrairDiagonal(int m[10][10], int diagonal[10], int n);
3 long produtoVetor(int v[], int n);
```

- `multiplicarMatrizes`: calcula o produto matricial $C = A \times B$ e armazena em `resultado`.
- `extrairDiagonal`: extrai os elementos da diagonal principal de uma matriz e os copia para um vetor.
- `produtoVetor`: calcula **recursivamente** o produto de todos os elementos de um vetor.

A função `main` deve ler duas matrizes A e B de dimensão N , calcular o produto $C = A \times B$ utilizando `multiplicarMatrizes`, extrair a diagonal principal de C em um vetor utilizando `extrairDiagonal`, calcular o produto dos elementos dessa diagonal utilizando `produtoVetor` e, por fim, exibir a matriz resultado C , o vetor da diagonal e o valor do produto calculado.

Questão 15

(Valor: 2,5 pontos) Uma indústria deseja desenvolver um **Sistema de Controle de Produção** em C para monitorar a quantidade de itens produzidos por diversas máquinas ao longo de vários dias. Os dados devem ser armazenados em uma matriz `producao`, em que cada linha representa uma máquina e cada coluna representa um dia.

Implemente as seguintes funções:

```
1 void calcularMediaMaquinas(int producao[20][30],
2                             int numMaquinas,
3                             int numDias,
4                             float medias[20]);
5
6 void calcularMediaDias(int producao[20][30],
7                        int numMaquinas,
```



```
8         int numDias ,  
9         float medias [30]);  
10  
11 void ordenarRanking(float medias [20] ,  
12                    int indices [20] ,  
13                    int numMaquinas);
```

- **calcularMediaMaquinas**: calcula a produção média de cada máquina;
- **calcularMediaDias**: calcula a produção média diária considerando todas as máquinas;
- **ordenarRanking**: gera um ranking das máquinas mais produtivas sem alterar os dados originais.

A função **main** deve ler o número de máquinas, o número de dias e os valores produzidos. Em seguida, o programa deve:

- a) calcular e exibir a produção média de cada máquina;
- b) calcular e exibir a produção média diária;
- c) gerar um ranking das máquinas em ordem decrescente de produção média;
- d) identificar a máquina mais produtiva.

Bom trabalho!